# COMP 3700A: Contributing to Open Source

# Continuous Integration and Deployment

Charlotte Curtis

November 2, 2026

# Topic Overview

- What problem is solved by CI/CD
- What is CI/CD
- Common implementations (YAML)
- CI/CD problems specific to FOSS

Primary reference: Martin Fowler on CI

# The problem

Imagine constructing a building:

- Plumbers, electricians, framers, drywallers, etc
- Things might work in isolation, but bringing them together (**integration**) introduces unexpected problems
- It's easier to integrate small changes than big ones

> *Just because everything works on your computer doesn't mean it will everywhere!*

# Continuous Integration

1. Make changes (feature, bugfix, etc)
2. Pull upstream changes
3. Resolve conflicts, build locally
4. Push changes
5. Automatic centralized CI service build and testing

> *Common constraint: anything merged into main must pass CI tests*



***What kinds of problems might be caught in CI that could be missed locally?***

# CI services

- GitHub/GitLab Actions
- CircleCI
- Jenkins
- Probably others

> *Most use YAML files to configure*



***How does CI/CD in FOSS differ from commercial software dev?***

# YAML Ain't Markup Language

- YAML is a file format to hold structured data
- Similar to JSON, but designed for human-readability

```
key: value # a comment
another-key:
  - a
  - list
  - ""of values # quotes not necessary but allowed
  nested-key: 0 # simple data types recognized
command: |
  multiline strings can be
  very useful
```

# GitHub actions example

```yaml
name: learn-github-actions
run-name: ${{ github.actor }} is learning GitHub Actions
on: [push]
jobs:
  check-bats-version:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-node@v7
        with:
          node-version: '24'
      - run: npm install -g bats
      - run: bats -v
```

from https://docs.github.com/en/actions/tutorials/create-an-example-workflow

# Continuous Delivery/Deployment

- The "D" in CI/CD can stand for **Delivery** or **Deployment**
- In order of increasing automation:
  - Integration: Builds and runs tests
  - Delivery: Prepares builds for release
  - Deployment: Releases to customers

> *What are some pros and cons of continuous deployment?*

# Release cycles and conventions

- CD build artifacts are often called **nightly builds**
- Common terms:
  - Alpha: Released for testing but may have serious bugs
    - (Feature freeze)
  - Beta: Addresses bugs found in Alpha, but still not stable
    - (String freeze)
  - Release Candidate (RC): should be basically ready
  - Final Release
- Various conventions for numbering or naming releases

# Semantic versioning

Specification described in https://semver.org

Given a version number `MAJOR.MINOR.PATCH`, increment the:

- `MAJOR` version when you make incompatible API changes
- `MINOR` version when you add functionality in a backward compatible manner
- `PATCH` version when you make backward compatible bug fixes

> *Example: Pandas*

# Calendar versioning

Less formally specified, but summarized at https://calver.org

- Still typically uses `MAJOR.MINOR`, but one or both components are based on the date (e.g. `YYYY.MM`)
- Only captures time between releases
- Best for fixed release cycles

> *Example: Ubuntu*



***What convention does your project use? Anything other than these two?***

# Containerization

> *A container is a standard unit of software that packages up code and all its dependencies so the application runs quickly and reliably from one computing environment to another. — Docker website*

- A solution to the problem of "but it works on my machine"
- **Not** the same thing as a virtual machine
- Commonly used for CI pipelines, development, deployed for web services

I'm going to rely on someone else' video overview

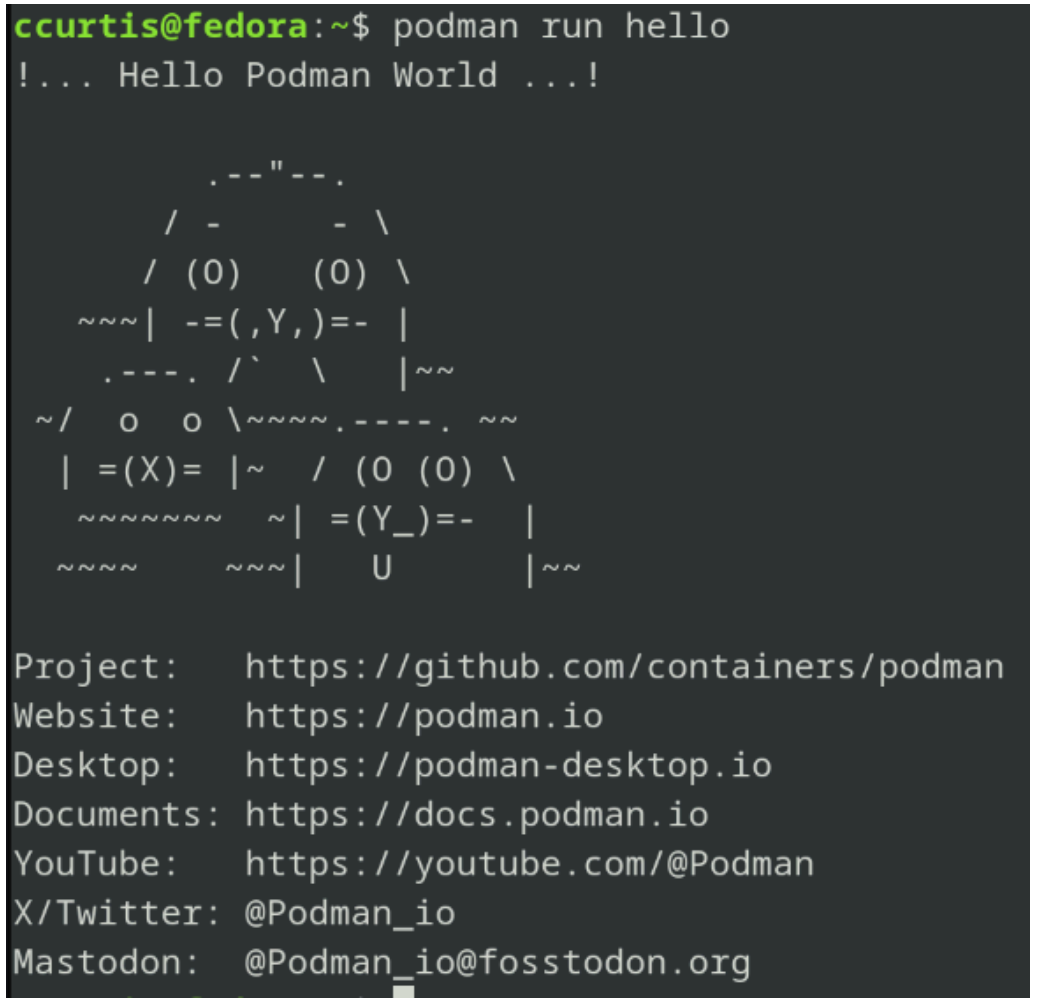
Originally scheduled as a separate lecture, but I think it fits here

# Implementations

- chroot is arguably the first "container" or "sandbox"
- Various other systems over the years
- Docker in 2013 has become a defacto standard
- Kubernetes (or K8s) was launched in 2014 to "orchestrate" Docker containers
- Podman was created in 2018 to be highly compatible with Docker, but daemonless, without root access by default

# Hello world

- The Hello world Docker image is as simple as it gets
- The source code looks kind of complicated though
- Podman is similar



# Additional reading

Here are some additional references that I used:

- Docker vs Podman YouTube short
- A Decade of Docker Containers for a more technical overview

# Coming up next

- Performing code reviews
- Unit testing
- Project part 2 presentations
- Lab: workflow exercise